

Práctica2: Taller de Coches Concurrente en Go

Sistemas Distribuidos

Grado en Ingeniería Telemática — URJC Raúl Molina López

Contenido

GitHub	1
Estructuras de Datos Principales	2
Funciones Principales del Sistema	3
Funcionamiento General	3
Diagramas UML	4
<i>Diagrama de secuencia Flujo Normal</i>	4
<i>Diagrama de secuencia Flujo Prioritario (>15s)</i>	4
Diagrama de casos de uso	5
Covertura de los test	6
Test 1 Duplicar cantidad de coches	6
Test 2 — Duplicar plantilla de mecánicos	6
Test 3 — Distribución de especialidades	7
Conclusión General de los Tests	7
Sección 3	8
Estructura de Archivos del Proyecto	8

GitHub

El hardware y los uml se encuentran en mi repositorio : <https://github.com/molilobo/distribuidosP2>

Estructuras de Datos Principales

A continuación se detallan las estructuras fundamentales implementadas en Go para modelar la concurrencia del taller:

- **Mecanico:** Representa a los operarios del taller encargados de las reparaciones.
 - **ID, Nombre, Especialidad** → Definen la identidad y competencia del mecánico (mecánica, eléctrica o carrocería).
 - **Libre bool** → Indicador de estado que determina si el mecánico está disponible para aceptar un trabajo.
 - **TrabajoCh channel** → Canal exclusivo a través del cual el mecánico recibe los coches asignados para atender.
- **Coche:** Representa los vehículos que ingresan al sistema requiriendo asistencia.
 - **ID, Incidencia** → Identificador único y tipo de avería en formato string.
 - **TiempoAcumulado int** → Contador de segundos acumulados en el proceso de reparación.
 - **Prioritario bool** → Indicador booleano que se activa (true) si el tiempo acumulado supera el umbral crítico de 15 segundos.
- **Taller:** Estructura global que coordina el ecosistema concurrente.
 - **Mecanicos []*Mecanico** → Slice que almacena la plantilla completa de mecánicos contratados.
 - **ColaCh chan *Coche** → Canal de comunicación que actúa como cola de espera sin límite para los vehículos entrantes.
 - **ResultadosCh chan string** → Canal de registro de eventos (log) que centraliza las notificaciones del sistema.

Funciones Principales del Sistema

<code>simularTaller()</code>	<code>main.go</code>	Lanza las goroutines base y se encarga del envío continuo de coches al sistema.
<code>dispatcher()</code>	<code>dispatcher.go</code>	Orquesta la asignación de coches pendientes en la cola hacia los mecánicos que estén libres.
<code>atenderCoche()</code>	<code>mecanico.go</code>	Goroutine individual por mecánico. Simula el proceso de reparación mediante suspensiones controladas (<code>time.Sleep</code>).
<code>tiempoAtencion()</code>	<code>mecanico.go</code>	Determina y devuelve los segundos de reparación requeridos según la especialidad del mecánico.
<code>contratarMecanico()</code>	<code>mecanico.go</code>	Permite la creación e incorporación en caliente de un nuevo mecánico al taller ante picos de demanda.

Funcionamiento General

El flujo de ejecución del taller concurrente sigue una arquitectura bien definida basada en pasos coordinados:

1. La función principal `main()` inicializa el entorno, da de alta a la plantilla inicial de mecánicos y arranca el `dispatcher()` como una goroutine en segundo plano.
2. Conforme los coches llegan de forma dinámica, son enviados directamente al canal `ColaCh`.
3. El `dispatcher()` realiza una lectura continua de la cola y busca activamente un mecánico libre que cuente con la especialidad requerida. Si no se encuentra disponibilidad, el coche permanece en espera en el canal.
4. Al asignarse el coche, cada mecánico procesa la tarea ejecutando `atenderCoche()` dentro de su propia goroutine independiente.
5. Monitoreo de Tiempos Críticos: Si se detecta que un coche acumula un `TiempoAcumulado > 15s` sin solventarse por completo, es devuelto inmediatamente a `ColaCh` con el flag `Prioritario` establecido en `true`.
6. Gestión de Cuellos de Botella: Si entra un coche con carácter prioritario y no existe ningún mecánico libre adecuado, el sistema escala de inmediato invocando `contratarMecanico()`, el cual añade un operario de refuerzo en caliente de dicha especialidad.

Diagramas UML

Diagrama de secuencia Flujo Normal

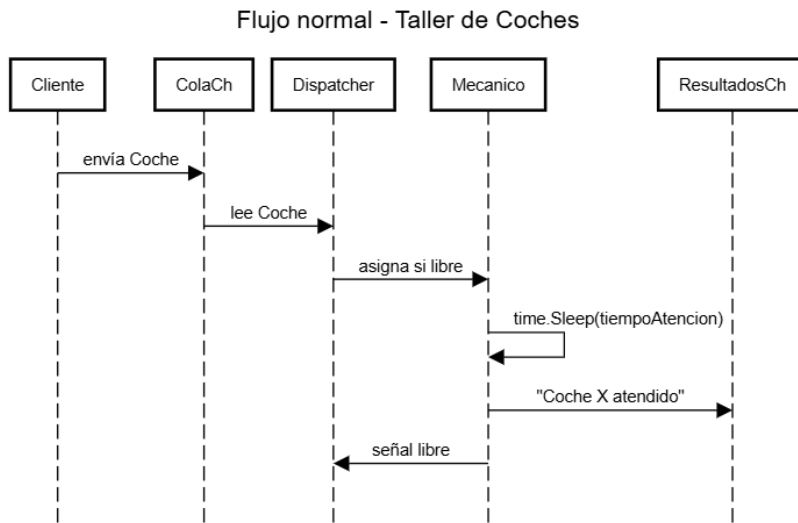


Diagrama de secuencia Flujo Prioritario (>15s)

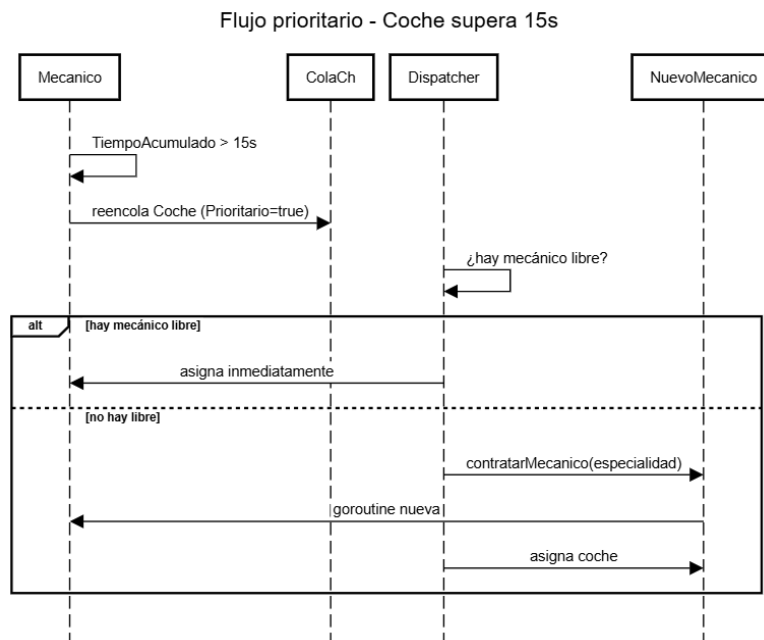
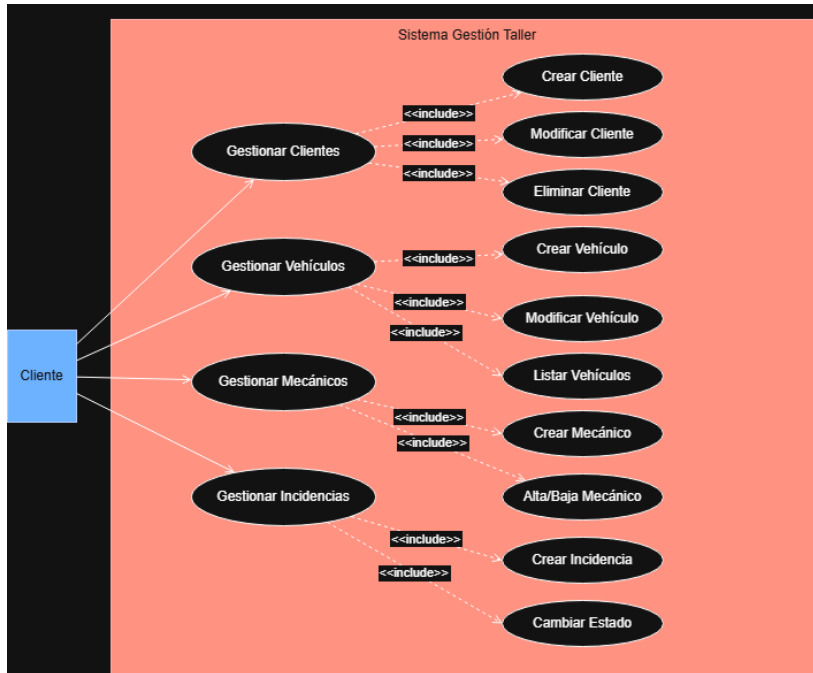


Diagrama de casos de uso

(deberia ser uno pero debido al tamaño y que este tipo de diagrama es simple, el como funciona taller puede no quedar bien reflejado)



Sección 2 -test

- Plantilla inicial: 3 mecánicos (1 Especialista en Mecánica, 1 en Electrónica y 1 en Carrocería).
- Tiempos de atención estandarizados: Mecánica = 5s | Electrónica = 7s | Carrocería = 11s.
- Umbral crítico de reencolado por prioridad: 15 segundos acumulados.

Las metricas de los test estan en txt en el repositorio

Covertura de los test

Debido a que los test no se exploran practicamente nada de la practica uno, al final no exploran el menu ni las distintas opciones esant centrados en los tiempos y que funcionen las go routines solo abracanm el 4% del codigo .

Test 1Duplicar cantidad de coches

Configuración	Coches	Tiempo Real Registrado
Base	N coches	~5s
Doble Carga	2N coches	~10s

El tiempo total de procesamiento se incrementa debido a que la plantilla de mecánicos permanece estática. Al introducir el doble de volumen de trabajo sobre los mismos recursos fijos, la velocidad de desalojo de la cola se satura, provocando que los coches experimenten un tiempo de espera proporcionalmente superior. Este comportamiento confirma que el sistema de almacenamiento en buffers de los canales (Go Channels) gestiona la carga de forma correcta y predecible bajo estrés.

Test 2 — Duplicar plantilla de mecánicos

Configuración	Mecánicos Activos	Tiempo Real Registrado
Base (3)	1 mecánica + 1 eléctrica + 1 carrocería	~21s
Doble Plantilla (6)	2 mecánica + 2 eléctrica + 2 carrocería	~11s

Análisis empírico: Al duplicar la cantidad de operarios concurrentes, el tiempo total se reduce prácticamente a la mitad. La desviación menor (no siendo la mitad exacta) se atribuye directamente al overhead de coordinación del planificador de Go, el bloqueo intermitente en los canales y la lógica de decisión interna del dispatcher(). Los resultados demuestran la alta eficiencia del paralelismo real logrado mediante el uso nativo de múltiples goroutines ejecutándose en paralelo.

Test 3 — Distribución de especialidades

Configuración	Composición de la Plantilla	Tiempo Real Registrado
A — Foco en Mecánica	3 mecánica + 1 eléctrica + 1 carrocería	~11s
B — Foco en Complejos	1 mecánica + 3 eléctrica + 3 carrocería	~11s

Sorprendentemente, ambas configuraciones arrojan el mismo tiempo final de ejecución. Esto pone de manifiesto la existencia de un cuello de botella crítico centrado en la especialidad de carrocería (cuyo coste unitario base es el más elevado: 11s por coche). En la configuración A, la carrocería continúa desasistida con un único mecánico lento. En la configuración B, a pesar de reforzar las áreas lentas, el tiempo unitario intrínseco sigue dominando la cola de procesamiento general. Esto comprueba una regla de oro en sistemas distribuidos: sobredimensionar componentes o recursos que no constituyen el cuello de botella actual no aporta mejoras al rendimiento global del sistema.

Conclusión General de los Tests

- Proporcionalidad de Carga: Ante un incremento de la carga de trabajo, el tiempo total escala linealmente siempre que los recursos de computación permanezcan fijos.
- Escalabilidad Horizontal: La inyección de más mecánicos al sistema escala el rendimiento de manera casi proporcional gracias a la óptima explotación del paralelismo de las goroutines.
- Optimización Selectiva: La distribución estratégica de especialidades solo altera favorablemente el resultado global si actúa directamente mitigando o eliminando el cuello de botella del sistema (en este caso, la especialidad de carrocería con 11 segundos).

Sección 3

Estructura de Archivos del Proyecto

Archivo	Contenido y Responsabilidad
<code>types.go</code>	Definición estricta de los modelos de datos compartidos: Mecanico, Coche y Taller.
<code>mecanico.go</code>	Contiene el núcleo lógico operacional de la atención: <code>atenderCoche()</code> , <code>tiempoAtencion()</code> y la rutina de escalado <code>contratarMecanico()</code> .
<code>dispatcher.go</code>	Implementación de la rutina central <code>dispatcher()</code> , encargada de sincronizar y emparejar la cola con los recursos ociosos.
<code>crud.go</code>	Módulos y funciones auxiliares enfocados en las tareas de gestión y operaciones administrativas internas.(practica1 adaptada ligeramente)
<code>main.go</code>	Punto de entrada de la aplicación. Configura las variables de entorno, inicializa las estructuras y arranca la simulación global <code>simularTaller()</code> .
<code>taller_test.go</code>	Batería de pruebas concurrentes automatizadas haciendo uso del paquete nativo <code>testing</code> de Go.